Physics Simulators: Time Steps, Contact and the Main Engines

Contents

1. Overview 2
2. How a simulator steps 4
   - 2.1 One step at a time 4
   - 2.2 Which velocity moves the position 4
3. Contact: what stops the box at the floor 6
   - 3.1 Finding the contact 6
   - 3.2 Hard contact: one impulse stops the fall 7
   - 3.3 Soft contact: sink, then pushed back 7
4. The time step: stable, jittery or unstable 9
5. Friction: holding, then slipping 11
6. The main engines and how to choose 14
7. Summary 17
8. Sources 18
9. Key terms 18

Where this fits:

- **Builds on:** Collision detection; Pushing and the quasistatic assumption; Second-order systems: natural frequency and damping ratio; Discretising a controller.

1. Overview

Key point: A physics simulator moves every body forward in small time steps: each step finds the contacts, adds up the forces, and updates the velocities and then the positions. The step size decides whether the result is accurate, jittery or unstable, and the contact and friction models decide how bodies touch and slide.

The robot can already plan and drive to the door, but only inside our own simulations, which move it exactly as commanded (the kinematic model, G-2297). The kinematic model has no masses and no forces, so it cannot say when a wheel slips, what happens if the robot bumps the table, or how a box it pushes slides. Robots are tested in physics simulators before they drive for real: Nav2 "provides out-of-the-box support for the Gazebo simulator ... for testing and evaluation before deploying new capabilities onto hardware robots" (Macenski et al. 2023 §IX; Nav2 start to finish), and MuJoCo and Isaac Sim are built for robot learning among other uses (MuJoCo docs: Overview; Isaac Sim docs: home).

Contact and friction are clearest on a single body, so this Note drops a box onto the room's floor before it returns to the robot's wheel:

- the box weighs 1 kg and is a 0.2 m cube, seen from the side;
- it starts with its bottom 0.1 m above the floor, moving sideways at 2.0 m/s;
- the floor's coefficient of friction is 0.4, as for RO-053's cup on the table.

By hand (Section 5), the box lands after 0.143 s, the landing takes 0.56 m/s off its sideways speed, and it slides to a stop 0.550 m from where it started, at 0.510 s. Figure 1 plays the same drop in our own small simulator, a Python model written for this Note (no engine such as MuJoCo runs here), with three different time steps. Watch the bottom row: the box should land and slide, but it is thrown 1.36 m into the air.

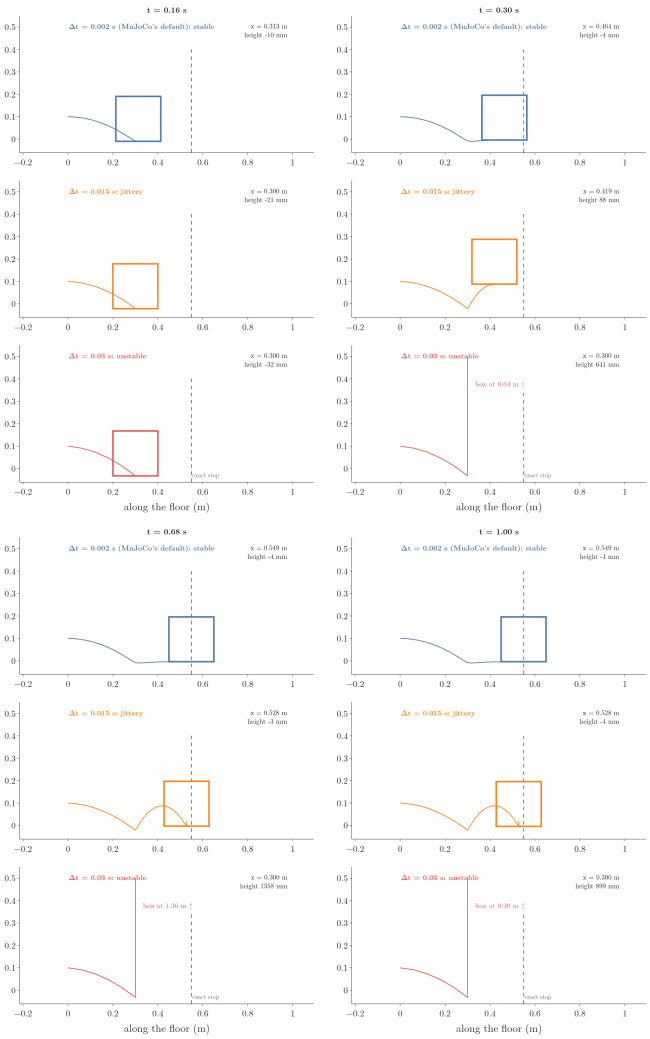


Figure 1: The running example in our own Python simulator (not MuJoCo or another engine) at three time steps, one frame per 0.02 s, all at the same scale: a 1 kg box (0.2 m cube) falls from 0.1 m with a sideways speed of 2.0 m/s onto a soft floor with friction coefficient 0.4. Top, a step of 0.002 s, MuJoCo's default: stable, stops at 0.549 m after 0.51 s, against the exact 0.550 m (grey dashes). Middle, 0.015 s: jittery, the box bounces 88 mm off the floor before it settles. Bottom, 0.03 s: unstable, the box is thrown 1.36 m up (an arrow shows its height when it leaves the frame)

The same model gives three answers: a stop at 0.549 m, a bounce of 88 mm, and a throw of 1.36 m (notebook). The Note treats the problem in five parts:

1. **step:** how a simulator moves the box forward in time (Section 2);
2. **contact:** what stops the box at the floor (Section 3);
3. **stability:** why the step size decides between the three answers (Section 4);
4. **friction:** what slows the slide, and when a wheel slips (Section 5);
5. **engines:** how Gazebo, MuJoCo, Isaac Sim and Drake do all this, and which to choose (Section 6).

2. How a simulator steps

> **Key point:** The first part of the problem, step: a simulator keeps each body's position and velocity, and at every time step finds the contacts, adds up the forces, turns them into an acceleration, and updates the velocity first and the position from the new velocity, which is semi-implicit Euler, MuJoCo's default.

Once the floor and friction act on the box, no closed formula gives its motion, so the motion must be computed in small steps.

2.1 One step at a time

The idea. We know how fast the box's speed changes at any moment: the forces on it, divided by its mass, by Newton's second law (G-2809). Holding that rate fixed over a short interval, we advance the speed and the position, compute the rate again, and repeat. Working out how a quantity evolves from its rate of change, step by step, is numerical integration of a differential equation; RO-001's Euler step (G-2299) did it for the robot's pose.

The standard terms. A program that computes how bodies move under forces, including their contacts and friction, by advancing their positions and velocities in small steps of time, is a **physics simulator** (G-3257), or physics engine. The fixed length of time it advances in one step is its **time step** (G-3258), written Δt; MuJoCo's default is 0.002 s, 500 steps per simulated second (MuJoCo XML reference). The rule that turns the rates of change into the new state at the end of a step is the **integrator** (G-3259).

MuJoCo's top-level function `mj_step` "does two things: compute the forward dynamics in continuous time, and then integrate over a time period specified by `mjModel.opt.timestep`", and a whole simulation is that function in a loop, `while (d->time < 10) mj_step(m, d);` (MuJoCo docs: Simulation). Our small Python simulator does the same four things in each step:

1. **find the contacts:** is the box's bottom at or below the floor, and how deep;
2. **forces to acceleration:** gravity, the floor's push and friction, divided by the mass;
3. **new velocity:** the old velocity plus acceleration times Δt;
4. **new position:** the old position plus the new velocity times Δt.

2.2 Which velocity moves the position

Step 4 could use the velocity from before the step or after it. With:

- z, the height of the box's bottom, in m, 0.1 at the start;
- v, its vertical velocity, in m/s, 0 at the start;
- g, the acceleration of gravity, 9.81 m/s^2;
- Δt, the time step, here a coarse 0.025 s so that each step shows;
- n, the step number, 0 at the start.

The first step of the fall, both ways:

$$v_1 = 0 - 9.81 \times 0.025 = -0.245$$

$$z_1 = 0.1 + (-0.245)(0.025) = 0.0939$$

$$z_1 = 0.1 + 0 \times 0.025 = 0.1$$

$$v_{n+1} = v_n - g\,\Delta t$$

$$z_{n+1} = z_n + v_{n+1}\,\Delta t$$

$$z_{n+1} = z_n + v_n\,\Delta t$$

The second line moves the position with the new velocity: the **semi-implicit Euler** (G-3260) rule. The third moves it with the old one, forward Euler, also called explicit Euler, under which the box does not move at all in its first step. The exact height after 0.025 s is 0.0969 m, between the two (notebook). MuJoCo does not implement explicit Euler, because semi-implicit Euler "is strictly better, and standard in physics simulation"; its default integrator, `Euler`, is this semi-implicit one, and its docs recommend `implicitfast` for most models (MuJoCo docs: Computation). With semi-implicit Euler, a force applied in a step already moves the body in that same step.

Figure 2 runs six steps. Watch the four parts light up in turn, and the blue points run ahead of the exact fall while the purple ones lag behind.

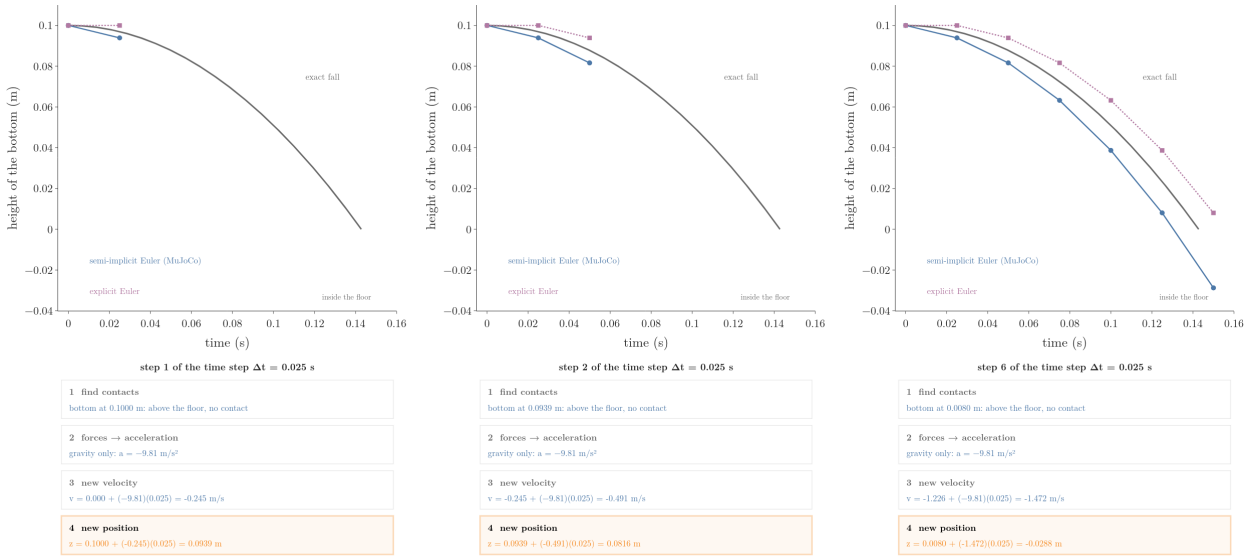


Figure 2: Our Python simulator on the falling box, one step at a time, with a coarse step of 0.025 s. Bottom: the four parts of a step lit in turn, with their numbers: find contacts (none while the bottom is above the floor), forces to acceleration (gravity only, -9.81 m/s^2), new velocity, new position. Top: the height of the box's bottom against time; grey, the exact fall, which lands at 0.143 s; blue, semi-implicit Euler (MuJoCo's rule); purple, explicit Euler. In step 6 the blue box ends 0.029 m inside the floor

In step 6 the box's bottom ends 0.029 m inside the floor: so far no part of the step accounts for the solid floor.

3. Contact: what stops the box at the floor

> **Key point:** The second part of the problem, contact: when a step would leave the box inside the floor, the simulator must push back. A hard contact stops the fall at once with an impulse and never lets the box in; a soft contact lets it sink a little and pushes back like a spring and a damper, which is what MuJoCo and Drake do.

Section 2's step put the box 0.029 m inside the floor, where no real box can be.

3.1 Finding the contact

The first part of each step asks whether the box touches the floor and how deep it is: the depth is how far its bottom lies below the floor's surface, 0 when it only touches. Deciding whether two bodies overlap is collision detection (G-2665); a simulator also needs the depth and the direction to push, the floor's upward normal here. The rule that turns a contact into forces is the simulator's **contact model** (G-3261). There are two kinds.

3.2 Hard contact: one impulse stops the fall

The idea. A ball thrown at a wall is stopped by a short, hard push. Over one step, what matters is the push times its time, the **impulse** (G-3264), in N s; by Newton's second law, an impulse changes a body's velocity by impulse divided by mass. With our steps of 0.002 s, in the step that would end inside the floor the box falls at 1.393 m/s (notebook). With:

- m, the box's mass, 1 kg;
- v, its downward speed into the floor, 1.393 m/s;
- J_n, the floor's impulse, upwards, in N s.

$$J_n = 1 \times 1.393 = 1.393 \text{ N s}$$

$$J_n = m\,v$$

After it, the box's downward speed is 0, and the simulator places it on the floor. A rigid-body solver does the same arithmetic for any contact: if one unit of impulse changes the contact's speed by 0.25 and the speed is -0.5, it applies two units (Gustafsson 2010). A contact model in which bodies never overlap, and each contact is stopped exactly by an impulse, is **hard contact** (G-3262).

3.3 Soft contact: sink, then pushed back

The idea. A rubber pad compresses slightly when the box lands, and pushes back harder the deeper and the faster it is compressed. So instead of an impulse, the floor pushes with a force from the depth and the sinking speed, like a spring and a damper side by side. MuJoCo works this way: "For soft contacts complementarity has to be violated: when there is penetration and the material is pushing the contacting bodies apart, both the normal force and velocity are positive" (MuJoCo docs: Computation). A contact model that allows a small overlap and pushes back with a force that grows with the depth and the speed of sinking is **soft contact** (G-3263).

We set our spring and damper from MuJoCo's default contact setting, `solref` "0.02 1": a time constant of 0.02 s, how quickly the contact responds, and a damping ratio (G-2992) of 1, damping that just stops it from swinging (MuJoCo docs: Modeling). MuJoCo's formula, with its impedance set to 1, gives per kilogram (MuJoCo docs: Modeling). With:

- τ, the time constant, 0.02 s;
- ζ, the damping ratio, 1;
- b, the damping, in N s/m;
- k, the stiffness, in N/m.

$$b = 2/0.02 = 100 \text{ N s/m}$$

$$k = 1/(0.02 \times 1)^2 = 2500 \text{ N/m}$$

$$b = 2/\tau$$

$$k = 1/(\tau\,\zeta)^2$$

The floor's push, with:

- d, the depth, in m;
- $\dot{d}$, how fast the depth grows, in m/s;
- k, the stiffness, 2500 N/m;
- b, the damping, 100 N s/m;
- N, the floor's push upwards, in N, never a pull.

$$N = \max(0,\ k\,d + b\,\dot{d})$$

Two checks on our numbers. In the first step of contact, the depth is 0.3 mm and the box sinks at 1.393 m/s, so the damper gives nearly all of the push:

$$N = 2500 \times 0.0003 + 100 \times 1.393$$

$$= 0.75 + 139.3 = 140 \text{ N}$$

At rest the push must equal the box's weight, which fixes how deep it sits:

$$2500 \times d = 1 \times 9.81$$

$$d = 0.0039 \text{ m}$$

$$d_{\text{rest}} = mg/k$$

Figure 3 lands the box both ways, one step of 0.002 s per frame. Watch the top row stop at the floor within one step, and the bottom row sink 9.7 mm, then rise back to 3.9 mm.

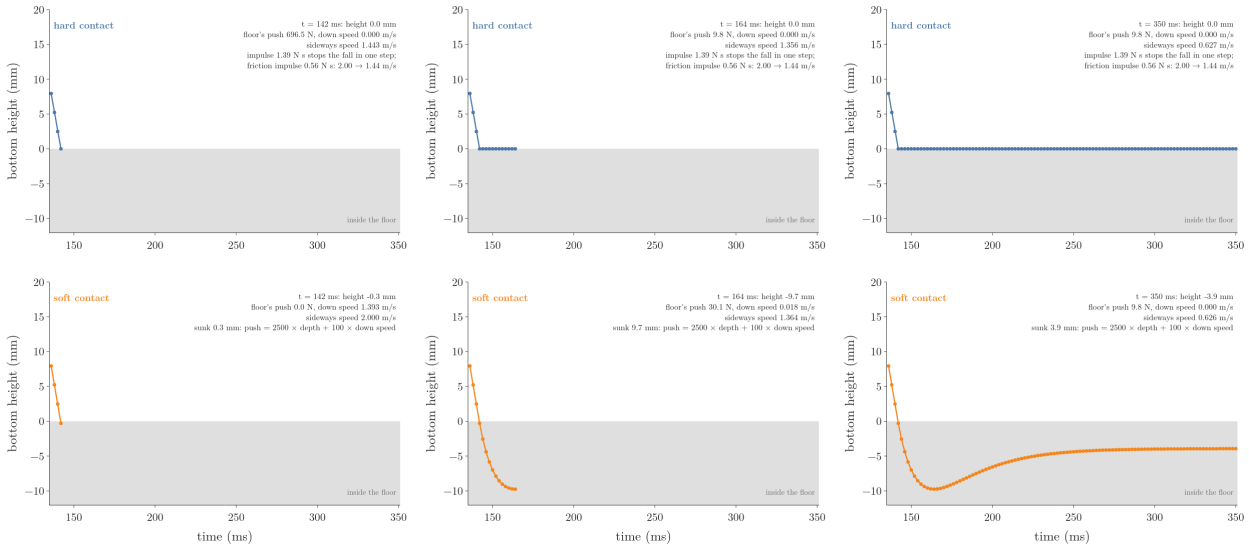


Figure 3: The landing of the running example in our Python simulator, one step of 0.002 s per frame from 136 ms to 200 ms, then one frame per 10 ms to 350 ms; the height of the box's bottom in millimetres, grey below 0 is inside the floor. Top, hard contact: in the step ending at 142 ms an impulse of 1.39 N s stops the fall and a friction impulse of 0.56 N s takes the sideways speed from 2.00 to 1.44 m/s; the box never enters the floor. Bottom, soft contact: the push reaches 140 N in the first step, the box sinks up to 9.7 mm at 164 ms and settles 3.9 mm deep, where the push equals its weight, 9.81 N

Both models give nearly the same slide: the box stops at 0.547 m with hard contact and 0.549 m with soft contact, against the exact 0.550 m (notebook). Soft contact costs a few millimetres of overlap; in return the force changes smoothly, which suits contact through soft materials. Drake's point contact is soft in the same way, with a `penetration_allowance` of 0.001 m by default, "the order of magnitude of the average penetration between bodies in the system that the user is willing to accept" (Drake docs: MultibodyPlantConfig, compliant contact). The spring and the damper resolve the contact, but they also raise the third part of the problem: the step size.

4. The time step: stable, jittery or unstable

> **Key point:** The third part of the problem, stability: in each step of contact the damper multiplies the sinking speed by one minus the damping times the step. With steps of 0.002, 0.015 and 0.03 s that factor is 0.8, −0.5 and −2: the speed shrinks smoothly, flips sign but shrinks, or flips and doubles, which is Figure 1's stable, jittery and unstable box.

Figure 1 used the same soft floor three times; only the step changed, and the answers were 0.549 m, a bounce of 88 mm and a throw of 1.36 m.

The idea. Consider the damper alone. In one step it takes away a fraction of the sinking speed: the damping times the step, divided by the mass. If that fraction is small, the speed shrinks a little each step. If it is more than 1, the step overshoots: the box is pushed back

faster than it was sinking. If it is more than 2, the overshoot is bigger than the speed it started from, and each step makes the error larger. With:

- v, the sinking speed at the start of a step, in m/s;
- b, the damping, 100 N s/m;
- m, the mass, 1 kg;
- Δt, the time step, in s.

The damper's part of step 3, on our three steps:

$$v_{\text{new}} = (1 - 100 \times 0.002)\, v = 0.8\, v$$

$$v_{\text{new}} = (1 - 100 \times 0.015)\, v = -0.5\, v$$

$$v_{\text{new}} = (1 - 100 \times 0.03)\, v = -2\, v$$

$$v_{\text{new}} = v - (b/m)\, v\, \Delta t$$

$$= (1 - b\, \Delta t/m)\, v$$

The factor stays between -1 and 1 only while:

$$b\, \Delta t/m < 2$$

$$\Delta t < 2 \times 1/100 = 0.02 \text{ s}$$

The three factors explain Figure 1:

- **0.002 s:** factor 0.8, the box settles smoothly;
- **0.015 s:** factor -0.5, the speed flips sign every step but shrinks, so the box chatters on the floor and bounces 88 mm before it settles;
- **0.03 s:** factor -2, the speed flips and doubles every step of contact, so each landing throws the box higher: 1.36 m, then 17 m on the next bounce (notebook).

The spring adds to the damper but does not change which of the three steps are stable (notebook). The energy that throws the box is not physical: the integrator's error creates it.

In real engines. MuJoCo's docs: "The time step is perhaps the single most important parameter that the user can adjust. If it is too large, the simulation will become unstable. If it is too small, CPU time will be wasted without meaningful improvement in accuracy" (MuJoCo docs: Computation). For its soft contacts they add: "The timeconst parameter should be at least two times larger than the simulation time step, otherwise the system

can become too stiff relative to the numerical integrator" (MuJoCo docs: Modeling), and MuJoCo enforces this internally unless its `refsafe` flag is turned off. For the default 0.02 s that allows steps up to 0.01 s, inside our 0.02 s limit with room to spare.

Keep the step fixed. A game or a viewer draws frames at whatever rate it can, but the physics must not take the frame time as its step: "your spring simulation exploding to infinity, fast moving objects tunneling through walls and players falling through the floor" (Fiedler 2004). Instead the simulator takes as many fixed steps as fit into the time that has passed and carries the rest over; with frames at 50 per second and physics at 100 steps per second, it takes two steps per frame (Fiedler 2004). Isaac Sim does the same: 60 physics steps per second by default, and "the simulation may run at a faster pace than rendering" (Isaac Sim docs: simulation fundamentals). With a stable step, the last part of the box's motion is the slide.

5. Friction: holding, then slipping

> **Key point:** The fourth part of the problem, friction: the floor's sideways force can take any value up to 0.4 times its push up, opposing the sliding. Below that limit the box holds; at the limit it slips. All the forces the floor can give lie inside a cone of half-angle 21.8°, the friction cone, and a robot's wheel grips while its push stays inside it.

The box landed sliding at 1.44 m/s; something must slow it to a stop at 0.550 m.

The idea. A heavy box pushed gently does not move: the floor's friction matches the push exactly. Above a certain push the floor can no longer hold the box, and it slides; while it slides, friction opposes the motion at that largest value. RO-053 gave the largest value: at most the coefficient of friction (G-2807) times the force pressing the surfaces together. The model in which the friction force opposes sliding and can be at most the coefficient of friction times the normal force, holding the box when it is enough and sliding at that value when it is not, is the **Coulomb friction model** (G-3265) (Lynch and Park §12.2). With:

- μ, the coefficient of friction, 0.4;
- N, the floor's push up, 9.81 N for the box at rest;
- f, the friction force along the floor, in N.

$$|f| \le 0.4 \times 9.81 = 3.92 \text{ N}$$

$$|f| \le \mu N$$

Holding, then slipping. Figure 4 pushes the box, at rest on the floor, with a force that grows by 2 N every second. Watch the friction match the push exactly until both reach 3.92 N, at 1.966 s; from then on the friction stays at 3.92 N, the push keeps growing, and the box speeds up to 1.08 m/s at 3.0 s (notebook).

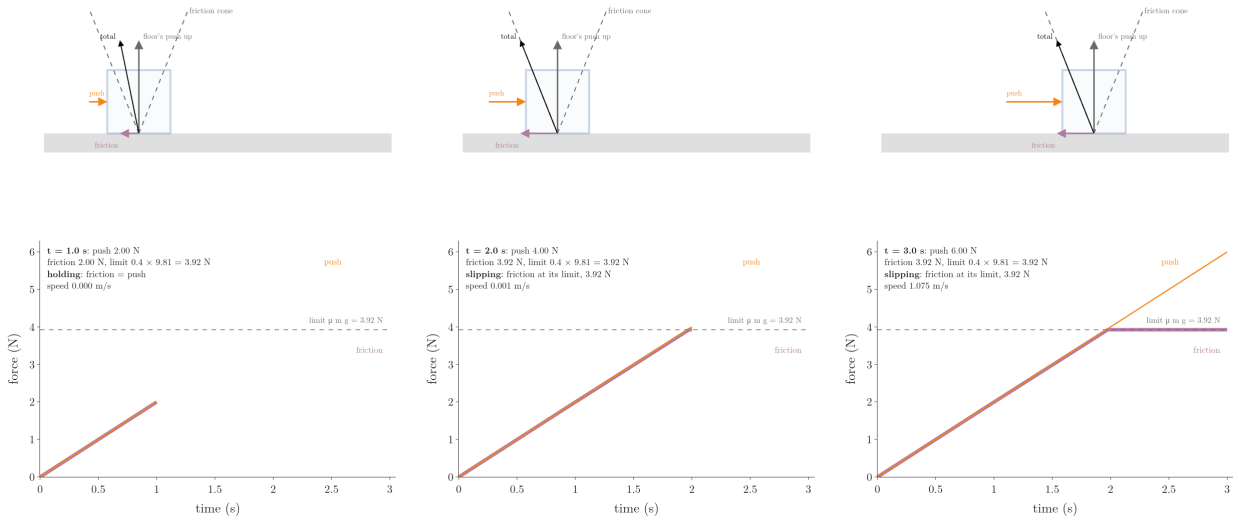


Figure 4: Friction holding, then slipping, in our Python simulator, one frame per 0.1 s (time step 0.002 s): the box at rest on the floor is pushed sideways with a force growing by 2 N per second (orange). Top: the push, the friction (purple), the floor's push up (grey), their total from the floor (black), and the friction cone (dashed), half-angle 21.8°; arrows drawn 0.03 m long per newton. While the box holds, the friction equals the push and the total lies inside the cone; from 1.966 s, when the push passes $0.4 \times 9.81 = 3.92$ N, the box slips and the total lies on the cone's edge. Bottom: push and friction against time

The friction cone. Add the floor's two forces, up and along, into one arrow. Because the sideways part is at most 0.4 times the upward part, the arrow can tilt at most this far from straight up. With:

- α, the largest tilt of that arrow from straight up, in degrees;
- μ, the coefficient of friction, 0.4.

$$\tan \alpha = 0.4$$

$$\alpha = 21.8^\circ$$

$$\tan \alpha = \mu$$

All the forces a contact can give make a cone round the normal, the **friction cone** (G-3266) (MuJoCo docs: Computation; Lynch and Park §12.2). Inside the cone the contact holds; on its edge it slips. MuJoCo builds the cone into its solver: `cone` "pyramidal" by default, an approximation by flat faces, while "Elliptic cones are a better model of the physical reality, but pyramidal cones sometimes make the solver faster and more robust" (MuJoCo XML reference).

In a step. Our simulator applies friction as an impulse (force times time, Section 3.2) in each step, capped at 0.4 times that step's normal impulse: if the cap is enough to stop the sliding, the box holds; if not, the full cap is taken off (Gustafsson 2010: "if the normal impulse is 2.0 and the friction coefficient is 0.9 your maximum friction impulse is 1.8"). The same cap explains why the landing slows the slide: in the landing step the normal impulse is 1.393 N s, so friction may take off up to:

$$0.4 \times 1.393 = 0.557 \text{ N s}$$

$$2.000 - 0.557 = 1.443 \text{ m/s}$$

The exact answer by hand. With a hard floor and steps shrunk to nothing, the drop has an exact answer: a fall, a landing that stops the fall and takes friction's share off the sideways speed, and a slide braked at 0.4 times gravity, whose length is the stopping distance at that deceleration. With:

- h, the drop height, 0.1 m;
- g, the acceleration of gravity, 9.81 m/s^2;
- μ, the coefficient of friction, 0.4;
- u_0, the sideways speed before landing, 2.0 m/s;
- t_1, the time of landing, in s;
- v_z, the downward speed at landing, in m/s;
- u_1, the sideways speed after landing, in m/s;
- s, the length of the slide, in m;
- x_{stop} and t_{stop}, where and when the box stops.

$$t_1 = \sqrt{2 \times 0.1/9.81} = 0.1428 \text{ s}$$

$$t_1 = \sqrt{2h/g}$$

$$v_z = 9.81 \times 0.1428 = 1.401 \text{ m/s}$$

$$v_z = g\, t_1$$

$$u_1 = 2.0 - 0.4 \times 1.401 = 1.440 \text{ m/s}$$

$$u_1 = u_0 - \mu\, v_z$$

$$s = 1.440^2/(2 \times 0.4 \times 9.81)$$

$$= 0.264 \text{ m}$$

$$s = u_1^2/(2\mu g)$$

$$x_{\text{stop}} = 2.0 \times 0.1428 + 0.264$$

$$= 0.550 \text{ m}$$

$$x_{\text{stop}} = u_0\, t_1 + s$$

$$t_{\text{stop}} = 0.1428 + 1.440/(0.4 \times 9.81)$$

$$= 0.510 \text{ s}$$

$$t_{\text{stop}} = t_1 + u_1/(\mu g)$$

Our simulator, with steps of 0.002 s, stops the box at 0.549 m (soft floor) and 0.547 m (hard floor), at 0.51 s (notebook).

The robot's wheel. The same rule decides whether the robot's wheels roll without slipping (G-2290). If the driven wheels carry the robot's whole weight on the same floor, the floor can push them forward with up to 0.4 times 9.81, 3.92 N per kilogram of robot. Speeding up at the room robot's 0.5 m/s^2 needs 0.5 N per kilogram (Newton's second law), about an eighth of that, so the push stays deep inside the cone and the wheels grip; asked to speed up faster than 3.92 m/s^2, they would slip (notebook). The simulator now reproduces the drop, the landing and the slide of Section 1; real engines perform the same four parts of a step with their own models.

6. The main engines and how to choose

> **Key point:** Gazebo, MuJoCo, Isaac Sim and Drake all step a state with an integrator (the rule from rates to the new state) and a contact and friction model, with different defaults: steps of 0.001 s, 0.002 s, 1/60 s and 0.001 s. We pick by what will run in it: a ROS 2 stack (Gazebo), contact-rich control and learning (MuJoCo), GPU physics with rendered sensors (Isaac Sim), or model-based design and verification (Drake).

Our simulator has one box and one floor; real engines run whole robots, and each makes its own choices for the four parts of a step.

Engine	Default step	Contact	Friction	Solver
MuJoCo	0.002 s, semi-implicit Euler	soft, time constant 0.02 s, damping ratio 1	pyramidal cone; sliding 1, torsional 0.005, rolling 0.0001	Newton, up to 100 iterations
Gazebo Sim	0.001 s (`max_step_size`)	DART engine; ODE collision detector	`mu` 1	Dantzig
Isaac Sim	60 steps per second	PhysX	per material	PGS or TGS
Drake	0.001 s (a discrete system)	hydroelastic, point contact as a fallback	Coulomb, stiction tolerance 0.0001 m/s	discrete approximation `lagged`

The sources of each row:

- **MuJoCo:** `timestep` 0.002, `integrator` Euler, `cone` pyramidal, `solver` Newton, `iterations` 100, `friction` "1 0.005 0.0001", `solref` "0.02 1" (MuJoCo XML reference). MuJoCo is "a general purpose physics engine that aims to facilitate research and development in robotics, biomechanics, graphics and animation, machine learning, and other areas that demand fast and accurate simulation of articulated structures interacting with their environment" (MuJoCo docs: Overview).
- **Gazebo Sim:** "Gazebo uses the DART physics engine by default", with DART's collision detector `ode` and solver `dantzig` as defaults, and Bullet as a preliminary alternative (Gazebo docs: physics engines); `max_step_size` 0.001 s, `real_time_factor` 1 and `mu` 1 are the SDFormat defaults (SDFormat 1.11). Its users "interact with running simulations from the outside — typically by publishing and subscribing to topics over Gazebo Transport or ROS" (Gazebo docs: Get Started).
- **Isaac Sim:** PhysX takes 60 steps per second unless the physics scene says otherwise, and can step more often than it renders (Isaac Sim docs: simulation fundamentals); PhysX offers two solvers, PGS and TGS (Omniverse physics docs). Isaac Sim lets us "Simulate with PhysX or Newton, add RTX and physics-based sensors, generate synthetic data, prepare robots for Isaac Lab, and validate robot stacks with ROS 2" (Isaac Sim docs: home).
- **Drake:** its plant settings, `MultibodyPlantConfig`, default to a time step of 0.001 s, hydroelastic contact with point contact as a fallback, the discrete approximation called "lagged", a penetration allowance of 0.001 m and a stiction tolerance of 0.0001 m/s (Drake docs: MultibodyPlantConfig). Hydroelastic contact, "rather than a single point, it imagines an entire contact surface" (Drake docs: compliant contact); "the discrete model is preferred for simulation given its robustness and speed in problems

with frictional contact" (Drake docs: MultibodyPlant). Drake describes itself as "Model-Based Design and Verification for Robotics" (Drake home page).

Figure 5 arranges the table as a choice, starting from what we will run.

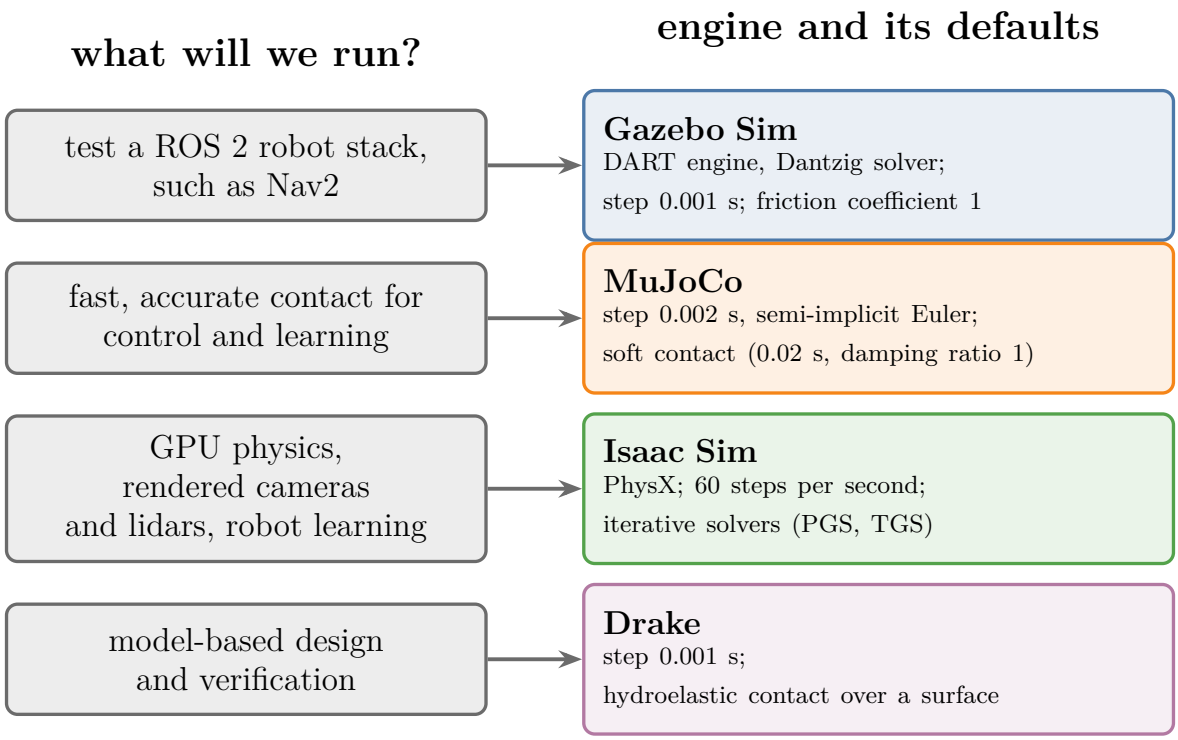


Figure 5: Choosing an engine by what we will run. A ROS 2 robot stack such as Nav2: Gazebo Sim (DART engine, Dantzig solver, step 0.001 s, friction coefficient 1). Fast, accurate contact for control and learning: MuJoCo (step 0.002 s, semi-implicit Euler, soft contact with time constant 0.02 s and damping ratio 1). GPU physics, rendered cameras and lidars, robot learning: Isaac Sim (PhysX, 60 steps per second, iterative solvers PGS and TGS). Model-based design and verification: Drake (step 0.001 s, hydroelastic contact over a surface). Defaults from each engine's documentation

Whichever engine runs, the same three checks apply to our room robot: a step small enough for its stiffest contact, a contact model whose overlap we can accept, and friction coefficients that match the real floor and wheels.

7. Summary

Part of a step	What it does	On the box
time step	the time advanced in one step	0.002 s stable, 0.015 s jittery, 0.03 s unstable
integrator	rates to the new state; semi-implicit Euler moves the position with the new velocity	first step: 0.0939 m, against 0.1 m for explicit Euler
hard contact	one impulse stops the fall, no overlap	1.393 N s; stops at 0.547 m
soft contact	spring and damper push back, small overlap	sinks 9.7 mm, rests 3.9 mm deep; stops at 0.549 m
Coulomb friction	sideways force up to μ times the push up	holds up to 3.92 N, then slips

- The dropped box should stop 0.550 m away at 0.510 s; with MuJoCo's default step of 0.002 s our Python simulator stops it at 0.549 m, while 0.015 s makes it bounce 88 mm and 0.03 s throws it 1.36 m, because the step size decides whether each step of contact shrinks or grows the motion.
- A **physics simulator** (G-3257) advances positions and velocities one **time step** (G-3258) at a time, finding contacts, adding forces and then integrating, because no formula gives the motion once bodies touch and slide.
- The **integrator** (G-3259) turns rates into the new state; **semi-implicit Euler** (G-3260), MuJoCo's default, updates the velocity first and moves the position with it, so a force acts in the same step: 0.0939 m after the first step, where explicit Euler leaves the box at 0.1 m.
- **Hard contact** (G-3262) stops the fall with one **impulse** (G-3264), 1.393 N s, so the box never enters the floor; **soft contact** (G-3263) pushes back like a spring and a damper, so the box sinks up to 9.7 mm and rests 3.9 mm deep, where the push equals its weight; both are **contact models** (G-3261) and give nearly the same slide.
- In each step of contact the damper multiplies the sinking speed by one minus 100 times the step: 0.8, -0.5 or -2 for our three steps, so with steps longer than 0.02 s the simulation diverges, and MuJoCo keeps its contact time constant at least two steps long.
- The **Coulomb friction model** (G-3265) lets the floor push sideways with up to 0.4 times its push up, so the box holds until 3.92 N and then slips; all those forces form the **friction cone** (G-3266) of half-angle 21.8°, and the robot's 0.5 m/s^2 needs only an eighth of it, so its wheels grip.
- Gazebo, MuJoCo, Isaac Sim and Drake make the same four choices with different defaults (steps of 0.001 s, 0.002 s, 1/60 s, 0.001 s), so the engine is chosen by what will run in it: a ROS 2 stack, contact-rich learning, GPU sensors, or model-based design.

We can now judge a simulator's answer: whether its step suits its contacts, and what its contact and friction models allow. One question remains for the room robot: how its local

planner chooses each command it sends, from a window of reachable speeds and turn rates, which is the dynamic window approach.

8. Sources

Built from

- winterdev, "Designing a Physics Engine in 5 minutes", YouTube, youtube.com/watch?v=-_IspRG548E: the order of a step (dynamics, collision detection, collision response), from its chapter list and frames
- MuJoCo documentation (stable, read 2026-10-12): "Computation" (integrators, soft contacts, friction cones, choosing the time step), "Simulation" (the simulation loop), "Modeling" (solver parameters, `solref`), "XML Reference" (defaults of `option` and `geom`), "Overview", https://mujoco.readthedocs.io (MuJoCo docs: Computation; Simulation; Modeling; Overview; MuJoCo XML reference)
- Gustafsson, D. (2010). "Explaining the rigid body solver". Blog post, blog.voxagon.se (Gustafsson 2010)
- Fiedler, G. (2004). "Fix your timestep!". Gaffer On Games, gafferongames.com (Fiedler 2004)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §12.2 "Contact forces and friction" (Coulomb friction, friction cones), http://modernrobotics.org (Lynch and Park §12.2)

Other references

- Gazebo Sim documentation, "Physics engines" (Gazebo Sim 9), gazebosim.org (physics) (Gazebo docs: physics engines); "Get Started", gazebosim.org/docs (Gazebo docs: Get Started)
- SDFormat specification 1.11, elements `physics` and `collision`, http://sdformat.org/spec (SDFormat 1.11)
- NVIDIA Isaac Sim documentation, home page and "Physics simulation fundamentals", https://docs.isaacsim.omniverse.nvidia.com/latest/ (Isaac Sim docs: home; simulation fundamentals); Omniverse physics developer guide, "Simulation control", docs.omniverse.nvidia.com (Omniverse physics docs)
- Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A. and Ferguson, M. (2023). "From the desks of ROS maintainers: a survey of modern & capable mobile robotics algorithms in the Robot Operating System 2". *Robotics and Autonomous Systems* 168, arXiv 2307.15236: §IX (Macenski et al. 2023)
- Drake documentation: `MultibodyPlantConfig`, `MultibodyPlant`, "Compliant contact", https://drake.mit.edu/doxygen_cxx/ (Drake docs: MultibodyPlantConfig; MultibodyPlant; compliant contact); home page, https://drake.mit.edu (Drake home page)

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Physics simulator (physics engine) (G-3257)	A program that computes how bodies move under forces, including their contacts and friction, by advancing their positions and velocities in small time steps, such as MuJoCo, Gazebo, Isaac Sim or Drake; robots and their software are tested in one before they meet the real world.
Time step (simulation) (G-3258)	The fixed length of simulated time a physics simulator advances in one step, such as MuJoCo's default 0.002 s; too large and the simulation becomes unstable, too small and computing time is wasted.
Integrator (simulation) (G-3259)	The rule a simulator uses to turn the rates of change of its state, velocities and accelerations, into the new state at the end of a time step, such as semi-implicit Euler or Runge–Kutta; it decides how accurate and how stable each step is.
Semi-implicit Euler (G-3260)	An integrator that first updates the velocity with the acceleration, $v_{k+1} = v_k + a\,\Delta t$, and then moves the position with the new velocity, $x_{k+1} = x_k + v_{k+1}\,\Delta t$; MuJoCo's default, it lets a force act on the position in the same step and is the standard in physics simulation.
Contact model (G-3261)	The rule by which a physics simulator turns a contact between two bodies, their overlap and how fast they approach, into forces or impulses that keep them apart; hard and soft contact are the two kinds.
Impulse (G-3264)	A force times the time it acts, in N s, for a short push such as a landing; it changes a body's velocity by impulse divided by mass, which is how hard contact and friction are applied within one step.
Hard contact (G-3262)	A contact model in which bodies never overlap: in the step a body would enter another, an impulse stops its approach exactly, such as 1.393 N s for a 1 kg box landing at 1.393 m/s; the force can jump from nothing to large in one step.

Term	Meaning
Soft contact (G-3263)	A contact model that lets bodies overlap a little and pushes them apart with a force that grows with the depth and the speed of sinking, like a spring and a damper; MuJoCo and Drake use it, so a 1 kg box rests a few millimetres deep and the force changes smoothly.
Coulomb friction model (G-3265)	The friction model in which the force along a contact opposes sliding and can be at most the coefficient of friction times the normal force, $\|f\| \le \mu N$: the contact holds while that is enough and slides with friction at that value when it is not.
Friction cone (G-3266)	The cone round a contact's normal that holds every force the contact can give under Coulomb friction, half-angle $\arctan \mu$, such as 21.8° for $\mu = 0.4$; a contact force inside it holds, one on its edge slips, and simulators build it into their solvers.
Kinematic model (G-2297)	Equations that give how a robot's configuration changes from its speed inputs alone, ignoring masses and forces, such as $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$; used to predict motion at low speed.
Newton's second law (G-2809)	The net force on a body equals its mass times its acceleration; it gives the force needed to change an object's speed, which the quasistatic assumption neglects.
Euler step (G-2299)	One step of predicting motion: hold the rates fixed for a short time Δt and add rate times Δt to each quantity; repeated, it traces a path, more accurately the smaller Δt is.
Collision detection (G-2665)	Deciding, yes or no, whether two bodies overlap, such as the robot's disk and the table; planners call it thousands of times instead of building the obstacle region.
Damping ratio (G-2992)	A second-order system's damping compared with the damping that just stops it from swinging, $\zeta = K_d/(2\sqrt{K_p I})$, 0.2 for D gain 0.08 and 1 for 0.4; it alone sets the shape of the response and its overshoot.

Term	Meaning
Coefficient of friction (G-2807)	The ratio of the largest friction force two surfaces can give to the force pressing them together, such as 0.4 for the cup on the table; it sets how hard a sliding object must be pushed.
Rolling without slipping (G-2290)	A wheel moving so that its contact point does not skid: it moves forward by its radius times the angle it turns and never slides sideways, which is what lets wheel turns predict motion.